Day - 2

27 October 2026 10:47

1. LCP
2. INP
3. CLS

LCP → Largest Contentful Paint

INP - Interaction to Next Paint

CLS - Cumulative layout shift.

User Browser
chrome | Firefox | Edge | Safari

↓

New Relic Brower Javascript Agent

↓

New Relic Browser Ingestion Endpoint

↓

NRDB

↓

Brower Monitoring Dashboard.

Real User monitoring.

Splunk - UBA

DT | DD - RUM

New Relic - BMD

Customer
POST | booking made

↓

CozyTravel Rest API

↓

External HTTPcalls.

Common HttpClient → localhost : 8092

1,801 ms

↓

Downstream Service
DB Processing

Booking req. Customer → entire journey

Trace ID : abc123

Trace id

Incoming span

SpanA : Booking Rest API (1808 ms)

SpanB : HTTP client [1608 ms]

↳ Span : Booking Backend [1708 ms]

↳ SpanD : Database [1200ms]

Banking fund Transfer

Multi span

Trace ID - ABC123

- ↳ S1 - API gateway
  - ↳ S2 - Authenticate
  - ↳ S3 - Transfer Service
    - ↳ S4 : Payment Call
    - ↳ S5 : DB Query

Customer
POST | transfer 50K

↓

API gateway 50ms
SPAN - S1

↓

Authentication Serv.
SPAN - S2 100ms

↓

Transfer Service
SPAN - S3 200ms

↓

Payment Service 1500ms

Trace ID - ABC123

Circle span TRACE ID : XYZ89

Single span — TRACE ID: XYZ89 — Span: Health check.

Span-55 → Payment Service SPAN-S4 → Database operation SPAN-S5

Unfragmented Trace:

S1 → API gateway
S2 → Transfer service
S3 → Payment service
S4 → DB query

Fragmented Trace:

Missing Telemetry
Dropped span
Incomplete Instrumentation

S1 → API gateway
S22 → Transfer service.
S3 → Payment Service span — Missing
S4 → DB query

S1 → S2 → S3 → S4 (F1 | F2)

Trace ID: ABC123

SPAN	SPAN ID	Parent SPAN ID	Status
API gateway	S1	None	Available
Booking service	S2	S1	"
Payment Service	S3	S2	"
DB Query	S4	S3	"

S1	S1	ABC123	S1 None.
↓	S2	ABC123	S2 S1
S2	S4	ABC123	S4 None.
↓			
?			

S4 → Unavailable Parent, Fragmented trace.

F1 → S1 → S2
F2 → S4

* JVM, Heap Memory, class count, G1 old gen, G1 eden space, Survivor space, GC, Thread, Http Session:-

Java application - Easy Travel, Rest APIs, Booking logic, Java object

JVM Runtime

Heap Memory	Survivor space
G1 eden space	
G1 old Gen.	
Non-Heap memory	
Metaspace	Code cache
Compressed class space	other JVM stuff
Thread Mgt.	GC
Worker Thread	Young GC
HTTP Pool	Concurrent marking
	Mixed GC

↓ APM Agent

Worker Thread | Mixed GC
HTTP Pool

↓

New Relic Java APM Agent

JVM Metrics → New Relic → Dashboard & Alert.

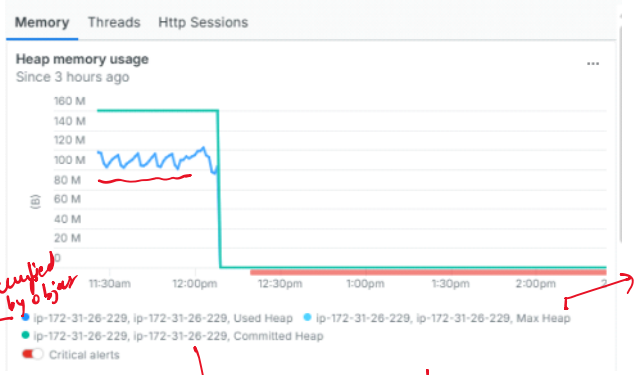
Measures — What?

1. Heap memory → Memory occupied by Java objects → Excessive Memory, frequent GC
2. Non-Heap Memory - JVM metadata & generated code memory → classloaders, Metadata growth
3. Class Count - No. of classes currently loaded → Unexpected class growth.
4. G1 old Gen - Long lived object → Retained objects, Memory pressure.
5. G1 Eden space - Most newly allocated object → High allocation rate.
6. G1 Survivor space - Object surviving for young collection → Object aging, Promotion
7. GC CPU Time - CPU effort spent on GC → Excessive GC
8. Threads - JVM thread activity & states → Blocked, deadlocked or excessive Threshold.
9. Http Session - Active Web Session → Session Accumulation, Memory Retention.

Heap Memory Usage:-

Customer Book Holiday

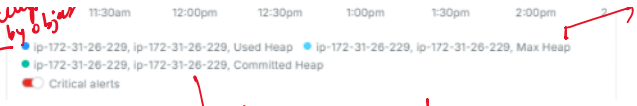
1. Customer object.
2. Booking object
3. Payment Details
4. Http req. object
5. DB result objects



Memory currently occupied by objects

Max Heap Size Allowed By JVM

Mem currently occupied by objects



↳ Memory JVM has obtained for Heap use.

Allowed By JVM -Xmx.

* G1 Eden, G1 survivor space, G1 old gen.



→ Object which will be still running after survivor space then it will be moved old gen.

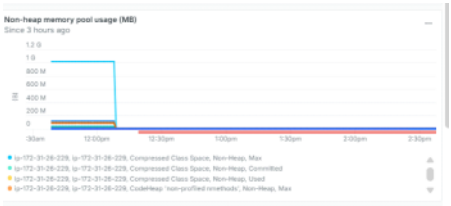


→ young gen.
→ New application objects are Allocated Here.



S0
S1
→ Object which are still Needed after Eden space. will be moved to survivor space.

* Non-Heap Memory & class Count:-



Metespace
Compressedclass
Code Cache.



→ Memory loaded & Unloaded Java classes.

* Garbage Collection CPU Time.

GC → Reclaim Memory occupied By object no longer Reachable.

* Thread Profiling:- Diagonstic Tool.

Application slow → CPU, Memory is Normal.

ET Java App.
↓
Background

Slow EasyTravel Booking.
→ Roofly

ET Java App.

HTTP Worker → DB Worker → Thread Profiler → Aggregated Call Tree

Background Thread.

Booking.

(1) 1.82 s → Booking

(2) Common HTTP | execute - 1.81 sec.

(3) Dept. - localhost:8091

(4) Http response: 200 OK

Java Thread.
↳ myTravelService.storeBooking Mode()
↳ CommonsHTTP.execute() → 65%
↳ SocketInputStream.read()

* Tomcat Worker Thread
↓
Booking Controller.
↓
Payment Http client
↓
request Read | waiting.

Top down → Call hierarchy
Bottom up → Frequently
↓
Hotspot

* Instrumentation:- Monitor the java classes & Method.
* Environment:- View JVM, OS, App. Env. details.
* Metric Normalization:- Rename | Ignore metrics using matching rules.

Before Normalization
Custom | Booking | 1001 → {id}.
Custom | Booking | 1002
Custom | Booking | 1003

After Normalization
Custom | Booking | {id}

Tomorrow's -
1. Browser Monitoring
2. Synthetic Monitoring
} + Week

Thus:-
1. NRQL → Event / Metric
Alert

For:-
(1) Report
(2) SLO | SLA |

SRE

SLI

New Relic

Query language.

(2.)

Alert

(2) Dashboard